

RELATÓRIO DO PROJETO DE TÉCNICAS ALGORÍTMICAS E COMPUTABILIDADE: ANÁLISE DE BENCHMARK ALGORÍTMICO

GUIÃO DA APRESENTAÇÃO ORAL

Unidade Curricular: Técnicas Algorítmicas e Computabilidade

Projeto: Benchmark do Desempenho de Algoritmos de Ordenação

Público-Alvo: Docente e avaliador Paulo Azevedo

1. INTRODUÇÃO METODOLÓGICA E ABERTURA

"Exmo. Professor Paulo Azevedo, vimos apresentar os resultados do nosso projeto de auditoria e benchmark de algoritmos de ordenação. O objetivo central deste estudo não é apenas rever as definições assintóticas tradicionais do âmbito estudado ao longo do semestre, mas sim analisar o comportamento empírico real destas estruturas quando submetidas a distribuições estatísticas complexas, num intervalo que vai de 10 a 10.000 elementos.

Em ambiente empresarial, a teoria pura do 'Pior' e 'Melhor' caso falha frequentemente em prever o comportamento de sistemas em produção. Por esse motivo, submetemos os algoritmos a perturbações probabilísticas controladas: ruído Gaussiano, assimetrias acentuadas via Distribuição Beta, e modelações de eventos independentes densos através de três variantes da Distribuição de Poisson. A nossa análise foca-se em como as restrições arquiteturais de cada algoritmo se traduzem visualmente nas curvas de tempo renderizadas nos nossos gráficos."

ARMAND

2. BUBBLE SORT: O IMPACTO DO CUSTO DE PERMUTAÇÃO

2.1. Complexidade

- **Melhor Caso:** $O(N)$
- **Caso Médio:** $O(N^2)$
- **Pior Caso:** $O(N^2)$

2.2. Defesa e Análise de Comportamento Gráfico

"Iniciamos o nosso painel analítico com o Bubble Sort. Quando projetamos o gráfico do Bubble Sort expandindo o volume de dados de 100 até 10.000 elementos, a curva temporal assume um perfil exponencial parabólico, que começa a sobrecarregar a CPU a partir do teto de 2.500 registos. Este fenómeno ocorre porque o Bubble Sort é penalizado pela operação física de *swap* — a troca de variáveis adjacentes na memória interna do computador.

Vejamos o comportamento do algoritmo mediante as diferentes distribuições:

- **Cenário Ordenado:** A nossa curva colapsa para uma linha horizontal rente ao zero absoluto ($O(N)$), provando a eficácia da otimização que o nosso grupo implementou.
- **Distribuições Gaussiana e Beta:** O gráfico sofre micro-oscilações. A Gaussiana, ao concentrar a densidade de dados em torno da média, cria micro-aglomerados ordenados locais, o que reduz marginalmente o número de trocas nas fases iniciais do ciclo.
- **Variantes de Poisson:** À medida que injetamos redundância e repetição de dados (passando de Poisson base para *Mais Repetidos* e *Muitos Repetidos*), notarão no gráfico que o tempo de execução estabiliza ligeiramente abaixo do cenário inverso absoluto. Isto deve-se ao facto de que elementos iguais saltam a validação condicional de troca, poupando o custo temporal interno da máquina, embora a barreira das comparações continue quadrática."

ARMAND

3. INSERTION SORT: A EFICIÊNCIA LOCAL E A SENSIBILIDADE À INVERSÃO

3.1. Complexidade

- **Melhor Caso:** $O(N)$
- **Caso Médio:** $O(N^2)$
- **Pior Caso:** $O(N^2)$

3.2. Defesa e Análise de Comportamento Gráfico

"O Insertion Sort tem uma ordenação incremental. No nosso benchmark, embora partilhe a mesma classe assintótica quadrática do Bubble Sort, o gráfico do Insertion Sort corre a uma velocidade superior, apresentando uma parábola mais baixa. Isso justifica-se pelo facto de o algoritmo substituir as trocas pesadas de variáveis por um mecanismo de deslocamento linear em cadeia, conhecido como *shifting*.

Analisando as curvas geradas, o Insertion Sort demonstra uma sensibilidade fascinante à entropia e à ordenação inicial dos dados:

- **Cenário Inverso:** A curva dispara verticalmente no Matplotlib. Cada novo elemento inserido exige que o ciclo interno caminhe retroativamente por todo o sub-vetor, maximizando o custo computacional.
- **Distribuições Beta e Gaussiana:** A distribuição Beta (com parâmetros $a=2$, $b=5$) gera uma assimetria com uma cauda longa à direita. No gráfico, isto traduz-se numa curva mais suave do que a da distribuição Uniforme. Como os dados possuem uma tendência direcional previsível, os elementos encontram o seu ponto de inserção muito mais cedo, poupando instruções de *shifting*.
- **Poisson com Muitos Repetidos:** A curva aproxima-se de uma resposta quase linear, pois o algoritmo encontra valores idênticos consecutivamente e interrompe o ciclo while interno quase de imediato."

ARMAND

4. SHELL SORT: A QUEBRA DA BARREIRA QUADRÁTICA VIA GAPS

4.1. Complexidade

- **Melhor Caso:** $O(N / \log N)$
- **Caso Médio / Pior Caso:** Depende estritamente da sequência de incrementos escolhida. Na nossa implementação (metade-metade: $\text{len}(A)//2$, $\text{len}(A)//4$, 1), o pior caso teórico fixa-se em $O(N^2)$, mas o caso médio empírico aproxima-se de $O(N^{\{1.25\}})$.

4.2. Defesa e Análise de Comportamento Gráfico

"O Shell Sort representa o primeiro grande salto tecnológico no nosso benchmark, quebrando conceptualmente a rigidez do teto quadrático através de comparações de longo alcance governadas por uma sequência decrescente de incrementos ou *gaps*.

Quando observamos a transição do volume de dados até aos 10.000 elementos no Matplotlib, o gráfico do Shell Sort já não exhibe aquela subida parabólica asfixiante dos métodos anteriores. A sua linha mantém-se incrivelmente estável, aproximando-se visualmente do comportamento dos algoritmos de elite (como o QuickSort).

O ponto de maior interesse científico na defesa deste algoritmo é a sua resiliência perante o cenário **Inverso**:

- Enquanto o Bubble e o Insertion colapsam no vetor inverso, o Shell Sort lida com ele de forma brilhante. As passagens iniciais com *gaps* grandes transportam os elementos massivos diretamente para o fim do array com pouquíssimos movimentos.
- Como consequência, a curva do cenário Inverso e as curvas das distribuições **Gaussiana** e **Uniforme** aparecem praticamente sobrepostas e aglutinadas no gráfico. O Shell Sort 'pré-ordena' a lista tão rapidamente que as pequenas variações de densidade estatística das distribuições ficam diluídas."

AFONSO

5. QUICK SORTs: PIVÔS E PARTIÇÕES

"Agora, passamos aos 3 tipos de Quick Sort estudados. Aqui, provamos que a escolha da estratégia do pivô e da geometria da partição dita se o algoritmo triunfa ou degenera. Analisámos 3 arquiteturas distintas."

5.1. Variante A: Quick Sort de Pivô Fixo (Partição de Lomuto)

- **Complexidade:** Caso Médio: $O(N / \log N)$ | Pior Caso: $O(N^2)$
- **Análise Gráfica:** "A curva do Quick Sort de Pivô Fixo é exemplar para demonstrar a natureza da degeneração algorítmica. Nas distribuições **Uniforme**, **Gaussiana** e **Beta**, o algoritmo exibe uma linha rasteira quase linear-logarítmica perfeita. No entanto, quando introduzimos os cenários **Ordenado** ou **Inverso**, a curva sofre uma mutação drástica: descola do chão e sobe de forma estritamente quadrática ($O(N^2)$). Como o pivô é fixo no primeiro elemento ($V[i]$), a partição falha em dividir o vetor ao meio, gerando sub-problemas desequilibrados de tamanhos 0 e N-1 que degradam a performance."

5.2. Variante B: Quick Sort com Pivô Aleatório (Partição de Hoare)

- **Complexidade:** Caso Médio e Pior Caso: $O(N / \log N)$
- **Análise Gráfica:** "Para mitigar a vulnerabilidade do pivô fixo, analisámos a variante com Pivô Aleatório junta ao esquema de partição bidirecional de Hoare. No Matplotlib, tem-se o efeito: as curvas dos cenários **Ordenado** e **Inverso**, que antes causavam uma "quebra" do sistema, colapsam de volta para o chão, fundindo-se numa linha de alta performance estável. A probabilidade matemática de escolher repetidamente um péssimo pivô através da função `random.randint` é estatisticamente desprezável, neutralizando os piores cenários."

5.3. Variante C: Quick Sort de 3 Partições (3-Way / Algoritmo de Dijkstra)

- **Complexidade:** Melhor Caso com chaves repetidas: $O(N)$ | Caso Médio: $O(N/\log N)$
- **Análise Gráfica:** "Esta variante foi desenvolvida de forma a “inovar” o QuickSort habitual: a redundância de dados. Reparem nas curvas das distribuições de **Poisson (mais rep)** e **Poisson (mais rep+)**. Nas variantes clássicas, dados repetidos forçam o algoritmo a efetuar partições desequilibradas sobre chaves iguais. No Quick Sort 3-Way, as curvas de Poisson com muitos elementos repetidos operam a uma velocidade assustadoramente rápida, ficando visivelmente **abaixo** de todas as outras distribuições contínuas. A partição tripla isola todos os duplicados no meio numa única passagem e remove-os das chamadas recursivas subsequentes."

GONÇALO

6. COUNTING SORT: A IMUNIDADE E A DEPENDÊNCIA DO EIXO K

6.1. Complexidade

- **Complexidade Global:** $O(N + K)$ (onde K é o valor máximo absoluto encontrado dentro do array)

6.2. Defesa e Análise de Comportamento Gráfico

"Concluímos a componente analítica com o único algoritmo não-comparativo do nosso projeto: o Counting Sort. O comportamento gráfico do Counting Sort tem uma lógica visual diferente de todos os anteriores. Enquanto os outros algoritmos mostram flutuações nas curvas quando mudamos da Gaussiana para a Beta ou para um vetor Inverso, a curva do Counting Sort é **rigorosamente imutável** face à organização inicial dos dados. O tempo de execução para um mesmo tamanho N é idêntico em todas as distribuições de dados, porque o algoritmo executa passos fixos obrigatórios de varrimento.

No entanto, o Counting Sort exhibe uma variação gráfica noutra vetor: o **teto do intervalo K**:

- O nosso código implementou com sucesso um wrapper matemático para transferir números negativos da distribuição **Gaussiana** para o eixo positivo.
- Contudo, reparem que na distribuição **Uniforme**, onde os dados são gerados no intervalo $n*10$, o valor de K cresce linearmente com o tamanho N ($K = 10N$). Isto faz com que o tamanho do array auxiliar C se estenda na memória.

- No gráfico final, o Counting Sort ganha de forma esmagadora em distribuições densas e contidas como as de **Poisson** (onde os números raramente passam de 15, tornando K constante e a curva puramente linear $O(N)$), mas sofre uma ligeira perda de inclinação na Uniforme devido ao custo de alocação de memória do espaço K expandido."

7. MATRIZ DE RESUMO ASSINTÓTICO PARA APRESENTAÇÃO

(Pode ser copiada diretamente como tabela para um slide ou folha de sumário do Word)

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Propriedade e Chave no Teste
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Altamente penalizado por trocas físicas (<i>swaps</i>).
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Eficiente em dados quase ordenados (Poisson).
Shell Sort	$O(N \cdot \log N)$	$O(N^{\{1.25\}})$	$O(N^2)$	$O(1)$	Neutraliza o cenário Inverso com saltos largos.
Quick Fixo	$O(N \cdot \log N)$	$O(N \cdot \log N)$	$O(N^2)$	$O(N/\log N)$	Degenera para quadrático em arrays já ordenados.
Quick Aleatório	$O(N \cdot \log N)$	$O(N \cdot \log N)$	$O(N/\log N)$	$O(N/\log N)$	Imune à ordenação prévia

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Propriedade e Chave no Teste
					devido ao pivô aleatório.
Quick 3-Way	$O(N)$	$O(N / \log N)$	$O(N / \log N)$	$O(N / \log N)$	O mais rápido em distribuições com muitos repetidos.
Counting Sort	$O(N+K)$	$O(N + K)$	$O(N + K)$	$O(N+K)$	Desempenho imutável; sensível apenas ao teto do valor K .

Conclusões do Projeto:

"Terminando a nossa apresentação, os gráficos de benchmark entregam-nos um veredito claro e acionável para o ambiente de desenvolvimento de software:

1. Para volumes de dados de larga escala com alta redundância de chaves (como logs parametrizados por distribuições de Poisson), o **Quick Sort de 3 Partições** e o **Counting Sort** (em intervalos controlados) são as soluções óbvias de alta performance.
2. O **Shell Sort** provou ser uma alternativa mais robusta e de baixo custo temporal, superando o Insertion Sort clássico por larga escala.
3. Algoritmos puramente quadráticos tradicionais, como o **Bubble Sort**, devem ser descontinuados de qualquer pipeline de dados que processe volumes acima de 1.000 registros.